Python OOP Foundations for AI

Target Audience: *AI Engineers & Developers*
Core Objective: *Mastering production-level Python for AI Orchestration and Agentic workflows.*

1. The Problem: Scalability in the AI Era

In modern AI development, you often handle complex data structures, such as managing 200,000+ employee records or complex agent states. Loose variables or simple dictionaries create a disconnect between data and behavior.

The Inefficient Approach (Loose Variables)

```
# Hardcoding - Unscalable and messy
emp_1_first = "Rahul"
emp_1_last = "Sharma"
emp_1_pay = 50000

emp_2_first = "Priya"
emp_2_last = "Patel"
emp_2_pay = 40000
```

The Functional Approach (Disconnected)

```
def create_employee(first, last, pay):
    email = f"{first.lower()}.{last.lower()}@infosys.com"
    return {"first": first, "last": last, "email": email, "pay": pay}

# Result is just a loose dictionary
emp_1 = create_employee("Rahul", "Sharma", 50000)
```

2. What is a Class? (The Blueprint)

A **Class** is a blueprint for creating objects. It allows you to bind data (attributes) and behaviors (methods) together in one logical unit.

- **Class:** The architectural blueprint of a building.
- **Object (Instance):** The actual physical building constructed from that blueprint.

3. The __init__ Method and 'self'

These are the core components of any Python class:

- **__init__:** A special method that runs **automatically** when you create a new object.
- **self:** A placeholder for the specific instance being created. It ensures data for one instance doesn't leak into another.

```
class Employee:
    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.email = f"{first.lower()}.{last.lower()}@infosys.com"

# Creating instances (Objects)
emp_1 = Employee("Rahul", "Sharma", 800000)
emp_2 = Employee("Priya", "Patel", 400000)
```

4. Instance Variables

Instance variables are unique to each object. Each instance maintains its own state independent of others.

```
# Updating only Rahul's pay
emp_1.pay = 850000

print(emp_1.pay) # 850000
```

```
print(emp_2.pay) # 400000 (Priya's pay remains unchanged)
```

5. Adding Methods (Behaviors)

Methods are functions defined inside a class that allow objects to perform actions.

```
class Employee:
    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay

    def full_name(self):
        return f"{self.first} {self.last}"

    def apply_raise(self, percentage):
        self.pay = int(self.pay * (1 + percentage / 100))

emp_1 = Employee("Rahul", "Sharma", 800000)
emp_1.apply_raise(4)
print(f"Salary after raise: {emp_1.pay}") # 832000
```

6. Why AI Engineers Must Master OOP

As an AI Engineer, OOP is your primary tool for building scalable systems:

- **Framework Standards:** Professional AI tools (LangChain, LangGraph, Vertex AI) are almost entirely class-based.
- **AI Orchestration:** To build "Agentic" workflows, you must define agents as classes that hold their own state and memory.
- **Debugging AI Code:** AI coding assistants often generate class-based code. You must understand the structure to maintain it.